

Resenha dos Capítulos 6 e 7 do livro Engenharia de Software Moderna

Nos capítulos 6 e 7 do livro "Engenharia de Software Moderna", são apresentados conceitos essenciais para o desenvolvimento de sistemas de software robustos e escaláveis, abordando tanto padrões de projeto quanto padrões arquiteturais.

No capítulo 6, que trata dos padrões de projeto, são discutidos dez padrões: Fábrica, Singleton, Proxy, Adaptador, Fachada, Decorador, Strategy, Observador, Template Method e Visitor. A ideia central é que "cada padrão descreve um problema recorrente em um determinado contexto e oferece uma solução que pode ser reutilizada inúmeras vezes." Para entender os padrões propostos por Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides, é fundamental compreender o problema que o padrão busca resolver, o contexto em que esse problema ocorre e a solução sugerida.

Cada padrão é discutido a partir de um contexto real, explicando o problema de design enfrentado e a solução oferecida por meio do padrão.

O padrão Fábrica surge em um contexto onde um sistema distribuído baseado em TCP/IP precisa criar diferentes tipos de canais de comunicação, como TCP e UDP. O problema é que o código não está preparado para ser facilmente estendido para suportar novos tipos de canais. A solução envolve utilizar o padrão Fábrica, que centraliza a criação de objetos por meio de um método estático, permitindo alterar o tipo de canal criado sem modificar o restante do código.

Já o padrão Singleton é aplicado quando um sistema precisa de uma única instância de uma classe para centralizar o registro de eventos. O problema enfrentado é que múltiplas instâncias dessa classe poderiam causar conflitos ao registrar eventos. A solução proposta é garantir que a classe tenha apenas uma instância, oferecendo um ponto global de acesso a ela.

O padrão Proxy aparece em um cenário onde um sistema de pesquisa de livros por ISBN deseja adicionar um cache para melhorar o desempenho, mas sem adicionar a lógica de cache diretamente na classe base. A solução envolve criar um objeto intermediário (proxy) que gerencia a lógica de cache, sem modificar a classe principal.

No caso do Adaptador, o contexto é um sistema que precisa controlar projetores de diferentes fabricantes, cada um com interfaces distintas. O problema é a falta de padronização nas interfaces dos projetores. O padrão Adaptador resolve essa questão ao converter a interface de um objeto existente para a interface esperada pelo sistema, permitindo que classes incompatíveis trabalhem juntas.

O padrão Fachada é utilizado quando um interpretador de linguagem possui várias classes internas que os usuários precisam conhecer para utilizá-lo. O problema é a complexidade do sistema, e a solução é oferecer uma interface simplificada que oculta a complexidade interna, facilitando o uso pelos usuários.

O padrão Decorador é aplicado em um sistema de comunicação que deseja adicionar funcionalidades opcionais, como buffers e compressão, a canais de comunicação. Usar herança para todas as combinações de funcionalidades geraria muitas subclasses, então o padrão Decorador permite adicionar funcionalidades dinamicamente, compondo objetos ao invés de usar herança.

O padrão Strategy é utilizado em um sistema de listas que deseja permitir a escolha de diferentes algoritmos de ordenação. O problema é que o código da classe de lista é inflexível, utilizando sempre o algoritmo Quicksort. A solução é encapsular diferentes algoritmos de ordenação, permitindo que sejam escolhidos dinamicamente.

No padrão Observador, o contexto é um sistema de estação meteorológica que precisa atualizar seus termômetros sempre que a temperatura muda. O problema é que não se deseja acoplar a classe de temperatura às classes de termômetros, já que as interfaces podem mudar. A solução é definir uma relação um-para-muitos, onde os observadores (termômetros) são notificados sempre que o sujeito (temperatura) muda.

O padrão Template Method surge em um sistema de folha de pagamento que precisa calcular o salário de diferentes tipos de funcionários. O problema é que o cálculo do salário segue uma estrutura padrão, mas varia conforme o tipo de funcionário. A solução envolve definir o esqueleto do algoritmo na classe base, permitindo que as subclasses implementem as partes específicas.

Por fim, o padrão Visitor é utilizado em um sistema de estacionamento que precisa realizar várias operações, como impressão de dados e persistência, sobre diferentes tipos de veículos. O problema é que adicionar essas operações diretamente nas classes de veículos criaria acoplamento e dificultaria a manutenção. O padrão Visitor permite adicionar novas operações a uma hierarquia de classes sem modificá-las, separando claramente a estrutura do comportamento.

Já no capítulo 7, é discutida a Arquitetura de Software, que é apresentada como uma série de decisões de alto nível que definem a estrutura e o comportamento de um sistema. A arquitetura não se limita apenas à organização de módulos ou componentes, mas também envolve decisões críticas, como a escolha de tecnologias e bancos de dados. Essas escolhas são difíceis de alterar posteriormente, o que as torna fundamentais para o sucesso e a manutenção do sistema ao longo do tempo.

O capítulo analisa diversos padrões arquiteturais, incluindo a Arquitetura em Camadas, que organiza o sistema em camadas hierárquicas e, mais especificamente, a Arquitetura em Três Camadas, amplamente

usada em sistemas de informação corporativos. Também é discutida a Arquitetura MVC (Model-View-Controller), que separa a aplicação em três partes: modelo, visão e controlador, facilitando a manutenção e escalabilidade. Além disso, o capítulo explora a Arquitetura baseada em Microsserviços, que propõe dividir grandes sistemas monolíticos em serviços menores e independentes, permitindo uma evolução mais rápida e uma melhor escalabilidade do sistema. Esses padrões arquiteturais são abordados como soluções para organizar e gerenciar sistemas complexos, garantindo que eles possam evoluir de forma controlada e eficiente.

Por fim, o texto alerta sobre os anti-padrões arquiteturais, como o "big ball of mud", que se refere a sistemas sem uma arquitetura clara e com um emaranhado de dependências, o que torna a manutenção extremamente difícil. O capítulo enfatiza a importância de um planejamento arquitetural adequado para evitar esses cenários.

Em conjunto, os capítulos destacam a importância de aplicar padrões de projeto e decisões arquiteturais de forma criteriosa, visando garantir flexibilidade, escalabilidade e fácil manutenção ao longo do tempo.